Documentation for Research Software

Building Software That Others (and Future You) Can Understand

Research Software Engineering

Why Documentation Matters

Onboarding

New team members can get up to speed without hours of hand-holding

Reproducibility

Others can replicate your results — a cornerstone of good science

Future You

Six months from now, you won't remember why you wrote that function

Collaboration

Open-source contributors and co-authors need to understand your code

Four Levels of Documentation

01

Comments & Docstrings

Inline explanations inside the code itself

02

README Files

The front door to your project

03

Guides & Tutorials

Narrative docs for complex projects

04

API Reference

Auto-generated from your docstrings

What Does This Code Do?

Take 60 seconds. Read the function below and try to figure out its purpose.

```python
def get_acc(p, t):
    if len(p) != len(t):
        return None
    if len(p) == 0:
        return 0.0

    c = 0
    for i in range(len(p)):
        if p[i] == t[i]:
            c += 1

    return c / len(p)
```

What do `p`, `t`, and `c` stand for? What does returning `None` silently do to downstream code?

The Documented Version

Same logic, now readable. Names explain what. Docstrings explain why.

```python
def calculate_accuracy(predictions: list, targets: list) -> float:
    """
    Calculates the accuracy of an AI model's predictions.

    This function compares the model's predicted labels against the
    actual correct labels (targets) and returns the ratio of
    correct predictions.

    Args:
        predictions (list): A list of predicted class labels from the model.
        targets (list): A list of the actual, correct class labels.

    Returns:
        float: The accuracy score, ranging from 0.0 (none correct)
               to 1.0 (all correct).

    Raises:
        ValueError: If the predictions and targets lists are not the same length.
        ZeroDivisionError: If the input lists are empty.

    Example:
        >>> preds = [1, 0, 1, 1]
        >>> actual = [1, 0, 0, 1]
        >>> calculate_accuracy(preds, actual)
        0.75
    """
    if len(predictions) != len(targets):
        raise ValueError("Predictions and targets must have the same number of items.")

    if len(predictions) == 0:
        raise ZeroDivisionError("Cannot calculate accuracy on empty lists.")

    # Count how many times the prediction matches the target
    correct_count = sum(1 for p, t in zip(predictions, targets) if p == t)

    return correct_count / len(predictions)
```

Clear naming

calculate_accuracy, predictions, targets tell you what they are instantly

Type hints

list -> float makes inputs and outputs explicit without reading the body

Docstring

A built-in description that help() and documentation tools can find

Loud errors

ValueError instead of silent None — bugs surface immediately

Level 1

What Is a Docstring?

A docstring is a string literal that appears as the first statement in a module, class, method, or function. Python stores it in the object's `__doc__` attribute, making it accessible at runtime via `help()` and documentation tools like Sphinx and pydoc.

Comment vs. Docstring

Comment #

```
# This calculates accuracy
def calculate_accuracy(...):
    ...
```

Ignored by Python. Only for human readers editing the source file.

Docstring """

```
def calculate_accuracy(...):
    """Calculates accuracy."""
    ...
```

Stored in __doc__. Accessible at runtime by help(), pydoc, Sphinx, and IDEs.

Anatomy of a Docstring

Google Style — the most common format in research Python code

```
"""
    Calculates the accuracy of an AI model's predictions.

    This function compares the model's predicted labels against the
    actual correct labels (targets) and returns the ratio of
    correct predictions.

    Args:
        predictions (list): A list of predicted class labels from the model.
        targets (list): A list of the actual, correct class labels.

    Returns:
        float: The accuracy score, ranging from 0.0 (none correct)
               to 1.0 (all correct).

    Raises:
        ValueError: If the predictions and targets lists are not the same
                    length.
        ZeroDivisionError: If the input lists are empty.

    Example:
        >>> preds = [1, 0, 1, 1]
        >>> actual = [1, 0, 0, 1]
        >>> calculate_accuracy(preds, actual)
        0.75
"""
```

Summary line

One sentence. What does it do?

Extended description

Optional. More context, caveats, or background.

Args

Each parameter: name, type, and a brief description.

Returns

Type and meaning of the return value.

Raises

Exceptions this function may throw.

Example

A runnable snippet showing basic usage.

Docstring Formats Compared

Pick one style and enforce it consistently across your project.

Google Style

```
"""One-line summary.

Args:
    x (int): Description.
    y (str): Description.

Returns:
    float: Description.

Raises:
    ValueError: If bad."""
```

NumPy / SciPy Style

```
"""One-line summary.

Parameters
----------
x : int
    Description.

Returns
-------
float
    Description."""
```

Google Style

More compact. Popular in ML/AI research. Used by TensorFlow, PyTorch, and most Google projects.

NumPy Style

More visual separation with underlines. Standard in scientific Python: NumPy, SciPy, pandas, scikit-learn.

Either works!

What matters is consistency. Pick one, document it in your contributing guide, and stick with it.

Comments & Docstrings

Best Practices

Do This

Explain why, not what
The code shows what — your comment explains intent

Use standard docstring formats
NumPy-style or Google-style for consistency

Document parameters & return values
Types, expected ranges, units if applicable

Include usage examples
A short example is worth a thousand words

Avoid This

 # Increment x by 1
Restating the obvious wastes space

Outdated comments
A wrong comment is worse than no comment

Leaving public functions without docstrings.
If it's public API, it needs documentation

Commented-out code blocks
Use version control, not comment graveyards

Level 2

README Files

The front door to your project — often the only documentation people read.

Project Name & Purpose

What does it do and why should I care?

Installation

Step-by-step setup, dependencies, system requirements

Quick Start / Usage

Minimal working example to get going fast

Configuration

Options, environment variables, config files

How to Cite

BibTeX entry, DOI, or citation file for academic use

License & Contributing

How to get involved, code of conduct, license info

Documentation Tooling

Pick the right tool for your ecosystem and complexity level.

Sphinx Python	The gold standard. Powers most major Python project docs. Supports autodoc, cross-referencing, and multiple output formats.	Best for: Large Python projects
MkDocs Python	Simpler setup with Markdown. The Material theme is excellent. Use mkdocstrings for API docs.	Best for: Mid-size projects
pdoc Python	Zero-config API doc generation. Reads your docstrings and builds a clean site automatically.	Best for: Quick API reference
Doxygen C/C++/Fortran	The standard for compiled languages. Generates docs from annotated source code comments.	Best for: HPC / compiled code

Documentation as Workflow

Don't write docs after. Write them during.

Docs-as-Code

Keep documentation in your repo alongside source code. Version, branch, and review it in pull requests.

Write as You Go

Document each function when you write it. If you wait, you'll forget the edge cases and design rationale.

Automate Building

Use CI/CD to build and deploy docs on every merge. Read the Docs and GitHub Actions make this painless.

Document Your Data

Data formats, config schemas, file naming — research projects fail when the data is undocumented.

Keep a CHANGELOG

Track what changed, when, and why. Vital for reproducibility and for collaborators joining mid-project.

Add CITATION.cff

Make your software citable. GitHub renders citation files directly — lower the barrier to getting credit.

Code Review as Documentation

Reviews are where documentation quality gets enforced — and where knowledge spreads.

Who

A senior reviewing a junior's PR may focus on style and system architecture. A newcomer may catch assumptions the team has stopped questioning. The audience shapes the review.

What

Single lines (pair programming), pull requests (code review), design docs (planning), test coverage (QA), or a full repo audit before release. Each scope needs different attention.

When

A suggestion to restructure data models is easy during design but impractical the week before release. Match the depth of feedback to where you are in the project lifecycle.

Why

To catch bugs a second pair of eyes can spot, to verify that changes are correct and necessary, to spread knowledge of the codebase, and to mentor new contributors.

Think of code review like editing a manuscript — you have knowledge that will help the code get better.

Developers Are People Too

Writing code is for machines. Writing reviews is for people.

Criticize the code, not the coder

✗ *"You should have used a dict here."*

✓ "We could consider using a dict here — it would speed up lookups from O(n) to O(1)."

Give reasons, not judgments

✗ *"It's stupid to use this algorithm."*

✓ "When sample size > 10k, a hash-based approach would be more memory efficient."

Give examples with context

✗ *"Use f-strings."*

✓ "f-strings are faster and easier to read here — something like: f\"Total: {count}\"."

Give feedback, not commands

✗ *"Rename this class to X."*

✓ "Can we rename this class to X? I think it better reflects its role as a data loader."

Give praise — reviews aren't just about mistakes. Recognize clever solutions and improvement.

Recognize stalemates — agree to disagree or bring in a third party. No one has to win every battle.

Questions & Discussion

"Code tells you how; comments tell you why."

— Jeff Atwood

Thank you!